SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Tejhash P	**Reg. No.:**	192512495
Section / Batch:	1	**Date:**	11/10/2026

Code of Conduct

I Tejhash P, 192512495 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Tejhash.P

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Answer

(a) Greedy Best-First Search (GBFS)

GBFS expands the frontier node with the lowest heuristic value, f(n) = h(n), i.e. the node whose straight-line distance to the destination is smallest. The drone would fly almost directly towards the hospital, expanding very few nodes, which gives a fast first route and low memory use.

- **Strengths:** very quick decisions, low computation and memory, and easy to re-run whenever a blocked path is reported, which suits urgency.
- **Weaknesses:** it ignores the cost already incurred g(n), so it cannot see that a short-looking route crosses bad terrain, strong wind or flooded land. It is not optimal and, without an explored set, not complete (it can loop or run into dead ends). Straight-line distance is misleading when a flooded river or blocked road forces a detour, so the drone may be drawn into a trap. It has no notion of risk, so under uncertainty it can commit to a route that is cheap by distance but unsafe.

(b) A* Search

A* orders the frontier by **f(n) = g(n) + h(n)**. Here g(n) is the actual cost of the path flown so far, using the variable terrain and weather costs (time and risk), and h(n) is the straight-line estimate of the remaining cost, scaled to cost units. The g term penalises routes that have become expensive, and the h term keeps the search pointed at the goal. If h never overestimates (admissible) and is consistent, A* is complete and returns the least-cost route. When a path is blocked that edge is removed and A* is re-run from the drone's current position. The price is more node expansions and memory than GBFS.

(c) Impact of heuristic accuracy

	Greedy best-first	**A***
Accurate h (close to true cost)	Near-direct, quick route	Few expansions, optimal route
Inaccurate or misleading h	Quality collapses: detours, dead ends, no bound on how bad the route is	Stays optimal if h is still admissible; only efficiency drops (more nodes)
Overestimating h (not admissible)	Same behaviour as above	May miss the cheapest route (loses optimality)
Weather-blind straight-line h	Ignores floods and wind entirely	Admissible if scaled by the minimum cost per km; adding weather multipliers tightens it

(d) Effect of dynamic changes (blocked paths)

- A computed plan can become invalid mid-flight, so the drone must **replan from its current position** each time updates arrive.
- GBFS replans cheaply but may repeatedly head for the same blocked area; A* replans at higher cost, but each new plan is optimal for the latest knowledge.
- Changed edge costs can break consistency, so closed nodes may need to be re-opened. Incremental methods (D* Lite, LPA*) reuse the earlier search instead of restarting, which keeps replanning time low.
- Updates are only partial: unknown segments should be treated as passable but with a risk penalty so the drone does not rely on stale information.

(e) Recommendation

Use A*, with risk-aware costs (flood, wind and blockage risk added to g), a safety margin on h, and **incremental replanning (D* Lite-style)**. A* gives the best balance of delay and risk, and because g includes risk it steers the drone away from hazardous terrain, which GBFS cannot do. To meet real-time limits, use **weighted A*** (f = g + w·h, w of about 1.2 to 1.5) or an anytime version that returns the best route found when the time budget ends, accepting a small, bounded loss of optimality. GBFS is kept only as an emergency fallback when a decision is needed immediately.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.
(b) Explain how Hill Climbing would work in this scenario and identify its limitations.
(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Answer

(a) Formulation as an optimisation problem and why classical search is inefficient

- **State (solution):** a vector x of timing parameters for all N intersections: cycle length, green split per phase, and offset for each junction.
- **Objective:** minimise $f(x) = w_1 \cdot$(total waiting time) + $w_2 \cdot$(fuel consumption) + $w_3 \cdot$(congestion index), evaluated from current traffic flow using a traffic model or simulator.
- **Constraints:** minimum and maximum green times, pedestrian crossing times, safe phase order, bounded cycle length.
- **Neighbourhood:** change one parameter of one intersection (for example +5 s green or a shifted offset).

Why traditional search is inefficient: there is no goal state or useful path, only a best configuration to find. The space is astronomically large (for 300 intersections with about 20 settings each for several parameters, 20^{300} or more combinations), so BFS, DFS or A* cannot store or enumerate it. Evaluating each candidate is expensive, neighbouring junctions interact (green waves), and the traffic pattern keeps changing so a one-off optimal plan becomes stale.

(b) Hill climbing and its limitations

Start from the current signal plan, generate neighbouring plans by small changes, move to the neighbour with the lowest cost (steepest-descent version), and repeat until no neighbour is better. It needs almost no memory, improves quickly, and can run continuously as traffic changes. **Limitations:** it gets stuck at local optima, wanders on plateaus, struggles on ridges, depends on the starting plan, never backtracks, and its optimum shifts as traffic changes.

(c) Local maxima, plateaus and ridges in traffic terms

Problem	Real-world meaning in this system
Local optimum (maximum of quality)	A plan where every single-junction change makes waiting time worse, yet a coordinated change would help; for example the current offsets are better than all neighbours, but a full green wave along a main road is not reachable by one tweak.
Plateau	A flat region where small changes in green time do not change waiting time, such as at night with low demand or when queues are saturated; hill climbing has no direction and may wander.
Ridge	A narrow path of good plans that single-parameter moves cannot follow: lengthening green on the arterial helps only if the downstream offset is shifted at the same time. Each single move looks worse, though the pair is better.

(d) How Simulated Annealing and Genetic Algorithms overcome these limitations

- **Simulated annealing:** pick a random neighbour; accept it if better, and if worse accept it with probability $e^{-\Delta E/T}$. At high temperature T it explores widely and escapes local optima, plateaus and ridges; as T is lowered it behaves like hill climbing and settles. It uses constant memory and can be *reheated* when traffic patterns change.
- **Genetic algorithm:** keep a population of complete timing plans (chromosome = all junction parameters), score them by fitness = −cost, and breed new plans by selection, crossover (for example swap the plans of whole corridors) and mutation. Many regions are explored in parallel, diversity avoids premature stalling, and crossover can combine good coordinated blocks, which helps on ridges. It needs many evaluations, but they can be run in parallel.

(e) Recommendation

Use **simulated annealing as the main method**, supported by a GA for offline design. Simulated annealing scales because it keeps only one candidate and evaluates one neighbour per step; it can be run per district in a decentralised way, and it adapts by reheating when congestion patterns shift. A GA is used offline to generate good city-wide baseline plans for typical regimes (morning peak, evening peak, night), which seed the online annealing. Plain hill climbing is kept only for quick local fine-tuning at the end.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.
(b) Analyze the challenges of partial observability and dynamic environments.
(c) Describe how the rover builds and updates its internal model.
(d) Compare online search with uninformed and informed search strategies.
(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Answer

(a) Why an online search agent is needed

An offline search agent computes the whole plan before acting and needs a known map and predictable results of actions. The rover has no map: it learns which actions are possible and what they lead to only by performing them. It must therefore **interleave planning and acting**. Earth–Mars communication delay (minutes each way) rules out remote step-by-step control, and some actions (falling into a crater) are irreversible, so the rover cannot try things freely. An online agent knows only the actions available in its current state, the step cost it just experienced, and whether it is at the goal.

(b) Challenges of partial observability and dynamic environments

- **Partial observability:** sensors see only nearby terrain; rocks and craters can be hidden by occlusion or shadow; hazard classification is noisy; position drifts without GPS, so even the rover's own location is uncertain.
- **Dynamic environment:** sand shifts, dust storms reduce visibility, lighting and temperature change, battery energy is consumed, so earlier map entries go stale.
- **Consequences:** dead ends and irreversible mistakes are possible, safe exploration is required, the agent can loop forever without memory of visited states, and onboard computing is limited.

(c) Building and updating the internal model

- **Map:** an occupancy-style grid where each cell is free, hazard or unknown, with a hazard probability and a traversal cost estimate.
- **Transition table** result[s, a] = s′ for every move already tried, a list of visited states, and unexplored actions per state (the basis of an online DFS agent, with a backtrack stack to return to earlier states).
- **Cost estimates** H(s), updated after each step as in LRTA*: H(s) = min over a of [c(s, a, s′) + H(s′)].
- **Cycle:** sense, update map and pose estimate (localisation and mapping), plan on the current model, take one step, repeat. New observations overwrite old ones, and uncertain cells keep a high cost.

(d) Online search compared with uninformed and informed search

Aspect	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online search (online DFS, LRTA*)
Knowledge needed	Full state space and action model	Full state space plus a heuristic	Only the current state's actions; map learned while moving
Planning vs acting	Plans completely before acting	Plans completely before acting	Interleaves planning and acting
Node expansion	Any node in memory	Any node in memory (jumps freely)	Only the node the rover is physically at
Direction	Blind; explores everywhere, wasting energy	Guided to goal by h	Guided by h, improved by learned costs
Safety	Offline: no real risk, but needs a map that does not exist	Offline: same	Must avoid unsafe moves; backtracking needs reversible actions
Optimality	BFS/UCS optimal on a known map	A* optimal with admissible h	No guarantee; judged by competitive ratio

(e) Most suitable approach

Use an **informed online search**: an LRTA*-style agent guided by a heuristic (straight-line distance to the sample site) whose cost estimates are learned, combined with **local replanning (D* Lite style) over the current map** after every sensor update. **Uncertainty** is handled by updating hazard probabilities and inflating costs for uncertain cells; **adaptability** comes from replanning each step; **safety** comes from refusing to enter cells whose hazard probability exceeds a threshold, preferring reversible moves, and

keeping a known safe path back. Offline A* is impossible without a map, and blind online DFS wastes energy.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Answer

(a) CSP formulation

- **Variables:** one per exam, E_c for each course c, with two parts: a time slot T_c and a hall H_c. (Invigilating faculty can be added as a third part.)
- **Domains:** T_c □ {1 … S}, all slots over the exam days (for example 3 sessions per day); H_c □ {all halls}. The domain of E_c is the set of (slot, hall) pairs.
- **Constraints:**

Requirement	Constraint	Type
No overlapping exams for any student	If courses c_1 and c_2 share a student then $T_{c1} \neq T_{c2}$	Binary
Limited halls	Exams in the same slot must use different halls: if $T_{c1} = T_{c2}$ then $H_{c1} \neq H_{c2}$; capacity(H_c) ≥ enrolment(c)	Binary, unary
Some subjects before others	For prerequisite order a before b: $T_a < T_b$	Binary
Faculty availability	Remove slots when the examiner is unavailable from the domain; one invigilator cannot be in two halls in one slot	Unary, binary
Special accommodations	Restrict domain to suitable halls (accessible, separate room) or longer slots; keep a rest gap between consecutive exams for those students	Unary, binary

(b) Backtracking search

Backtracking assigns variables one at a time in depth-first order. Pick an unassigned exam, try a (slot, hall) value from its domain, and check it against the constraints with exams already assigned. If it is consistent, go deeper; if every value for a variable fails, **backtrack** to the previous exam, undo its assignment and try its next value. The search ends when all exams are scheduled (a solution) or all options at the root are exhausted. Example: CS101 gets slot 1 in Hall A; MA101 shares students so it cannot take slot 1 and gets slot 2; later an exam finds no legal slot, so the search returns to an earlier exam and changes it. Ordering heuristics help: choose the most constrained exam first (MRV), then the one with most conflicts (degree), and try the least constraining value first. Its weakness is that failures are found late, so it can thrash through large parts of the tree.

(c) How forward checking improves efficiency

After each assignment, forward checking removes values that are now inconsistent from the domains of the neighbouring unassigned exams (those sharing students, prerequisites or the same halls). If any domain becomes empty, the search **backtracks immediately** rather than discovering the dead end many levels later. Example: after CS101 = slot 1, slot 1 is deleted from every exam sharing a student with CS101; if Physics is left with no slot, that branch is cut at once. With MRV, the exam with the smallest remaining domain is picked next, which makes the search “fail first”. Stronger propagation such as arc consistency (AC-3 / MAC) catches more conflicts. The extra bookkeeping per step is repaid by far fewer nodes explored.

(d) Real-world challenges and best strategy for scalability

- **Size:** thousands of courses and students give a very large conflict graph, so plain backtracking becomes too slow.
- **Over-constrained problems:** a fully valid timetable may not exist, so soft constraints (avoid consecutive exams, fair spacing) need weights and relaxation.

- **Change:** late registrations, hall closures or faculty leave require repair, not a full re-solve.

Best strategy: a hybrid. Solve the hard constraints with **backtracking + MRV/degree/LCV ordering + forward checking or AC-3**, and split the problem using the structure of the conflict graph (independent departments or components solved separately). Then use **min-conflicts local search** to improve soft preferences and repair the timetable when data changes. This keeps solutions valid while scaling to large institutions.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.
(b) Analyze the computational challenges of large game trees.
(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
(d) Design an evaluation function and justify the factors considered.
(e) Recommend a strategy for balancing optimality and real-time performance.

Answer

(a) Minimax model

The game is modelled as an adversarial search tree with an initial state, ACTIONS(s), RESULT(s, a), a terminal test and a utility. The AI is **MAX** and the human is **MIN**, and they alternate moves. Terminal or depth-limited leaves are scored (win/loss, or an evaluation). Values are backed up: a MAX node takes the **maximum** of its children and a MIN node the **minimum**, assuming the opponent plays optimally. At the root the AI picks the move with the best backed-up value, i.e. the best guaranteed outcome against the strongest reply. Time is $O(b^m)$ and space $O(bm)$ for branching factor b and depth m.

(b) Computational challenges of large game trees

- **Branching factor** is huge (many units each with many actions), and **depth** is large; for example b = 30 and depth 10 already gives about 6×10^{14} nodes.
- Strict **time limits** per decision leave time for only a shallow search.
- Each leaf needs a costly state evaluation; the **horizon effect** can hide threats just beyond the search depth.
- Real-time strategy games add simultaneous moves, hidden information (fog of war), randomness and continuous space and time, which break the assumptions of pure minimax.

(c) Alpha-Beta pruning

Alpha-beta keeps two bounds while searching: **α**, the best value MAX can already guarantee on the current path, and **β**, the best value MIN can already guarantee. A branch is pruned as soon as **α ≥ β**, because the remaining children cannot change the decision at the root.

- **At a MIN node** whose value so far is ≤ α: MAX already has a better alternative elsewhere and will never choose this node, so MIN's remaining children are skipped.
- **At a MAX node** whose value so far is ≥ β: MIN above already has a better option, so this node is never reached and its remaining children are skipped.
- **Example:** the root MAX has two MIN children. The first child's leaves 3 and 5 give value 3, so α = 3. In the second child the first leaf is 2, so its value is ≤ 2 < 3; the other leaves are pruned without being seen.
- **Result and cost:** it returns exactly the same move as full minimax. With the worst move ordering nothing is pruned ($O(b^m)$); with the best ordering the work drops to about $O(b^{m/2})$, roughly doubling the depth reachable in the same time. Good ordering (best moves first, killer moves, results from the previous iterative-deepening pass) is therefore important.

(d) Evaluation function design

Use a weighted linear function applied at the depth limit: **Eval(s) = $w_1f_1 + w_2f_2 + \ldots + w_nf_n$**, scored from the AI's point of view.

Feature	Meaning	Why it is included
f_1 Army strength	Value-weighted units and health of own minus enemy forces	Winning fights decides games
f_2 Economy	Resource income, workers, expansions	Resources convert into future strength
f_3 Map and vision control	Territory, chokepoints, high ground, scouting coverage	Position gives tactical advantage and information
f_4 Tech and production	Buildings, upgrades, unit production capacity	Long-term power and flexibility
f_5 Safety and threats	Base health, enemy distance to own base, exposed units	Avoids losing suddenly
f_6 Tempo	Time advantage, idle production	Initiative matters in real time

Features are normalised (for example own/enemy ratios) so they are comparable, must be cheap to compute because they run at every leaf, and are applied only at quiet positions (no battle in progress). Terminal wins and losses get ±∞. Weights are tuned from expert knowledge or by self-play and learning.

(e) Strategy balancing optimality and real-time performance

Use **iterative-deepening alpha-beta search** with a time budget: search to depth 1, 2, 3 … and when time runs out play the best move from the last completed depth (an anytime approach). Improve it with **move ordering** and a **transposition table** to avoid re-searching repeated states, and reduce the branching factor by **abstraction** (treating units as squads with high-level actions) and forward pruning of obviously poor moves (beam search). Use the evaluation function from (d) at the horizon, and where information is hidden or simultaneous, combine with **Monte-Carlo Tree Search** samples. Optimality is thus guaranteed only up to the search horizon, which is the sensible trade-off for real-time play; a quick scripted rule move is the fallback if time is nearly gone.